

Gemini / Antigravity Build Prompt

AetherPact MVP, Real AI, Netlify + Render Deployment

Version 1.0 | Paste the full prompt block below directly into Gemini Flash on Antigravity.

Scoping Notes, Read Before Pasting

A few deliberate scoping calls were made to keep this build honest and finishable in about 30 minutes. Worth knowing before you paste the prompt, and worth repeating to judges in exactly these terms if asked.

- **Matching uses TF-IDF plus cosine similarity (scikit-learn), not the MiniLM transformer from the full PRD.** MiniLM needs torch, which is a large, slow install that risks blowing both the 30-minute build window and Render's free-tier build limits. TF-IDF is real, fast, dependency-light, and genuinely reflects text similarity, just with less semantic depth than a transformer model.
 - **The LLM negotiation transcript (Qwen) is dropped for this build.** The clearing price is still 100 percent real math, computed by the same Zone-of-Possible-Agreement solver from the PRD. The settlement summary shown in the UI is an honest, clearly non-AI templated sentence built from the actual computed numbers, labeled as "Automated Settlement," not passed off as AI-generated text.
 - **Visual verification (Tier 3 from the PRD) is out of scope for this build entirely.** There was not enough time to implement it for real, so it is better left out of the demo than faked.
 - Every score, ranking, and price shown in the finished site is computed live from real user input. Nothing is hardcoded, randomized, or faked anywhere in this build.
-

The Prompt

Copy everything inside the code block below, exactly as it appears, and paste it into Gemini Flash on Antigravity as a single message.

PROJECT: AetherPact – B2B Hospitality Resource Matching MVP (real, working demo)

CONTEXT AND NON-NEGOTIABLE RULE:

This is a real working prototype for a hackathon demo, deployed publicly (frontend on Netlify, backend on Render or Hugging Face Spaces). Every score, ranking, and price shown in the UI **MUST** be computed live from actual user input using real logic defined below. Never hardcode, randomize, or fake a result under any circumstance, even under time pressure. If a feature can't be finished for real in time, cut the feature – do not replace it with a fake version. A smaller feature set that is 100% real is the goal, not a larger one that isn't.

=====

TECH STACK

=====

- Backend: Python, FastAPI, scikit-learn, uvicorn. No torch, no heavy ML libraries.
- Storage: in-memory Python data structures (a list of dicts). No database needed for this demo.
- Frontend: plain HTML, CSS, and vanilla JavaScript. No React, no build step, no npm – a single static site that can be deployed to Netlify by dragging the folder in or connecting a git repo, with zero risk of a build pipeline failing.
- Backend host: Render.com (free tier) using a simple Procfile/start command, or Hugging Face Spaces (Gradio/FastAPI SDK) as a fallback if Render's free build is slow.

=====

BACKEND – build this first, exactly as specified

=====

Folder structure:

```
backend/  
  main.py  
  requirements.txt  
requirements.txt:
```

fastapi
uvicorn
scikit-learn
python-multipart

main.py must implement:

1. CORS middleware allowing all origins (for the hackathon demo; note in a comment that this should be restricted in production).
2. In-memory data store:
listings = [] # each: {id, title, description, resource_type, price, capacity, lat, lng}
Seed this list at startup with 5 realistic hospitality listings, for example:
 - "Grand Ballroom, Hotel Sunrise" – banquet hall, capacity 200, price 15000, description mentioning banquet hall, weddings, events
 - "Commercial Kitchen, CloudBite Kitchens" – kitchen, description mentioning deck ovens, catering, commercial kitchen space
 - "AV Equipment Set, EventPro Rentals" – AV equipment, description mentioning speakers, projectors, lighting
 - "Shuttle Vans, Resort Meridian" – vehicle, description mentioning vans, guest transport
 - "Rooftop Terrace, Hotel Bayview" – banquet hall, description mentioning rooftop, outdoor event spaceGive each a distinct lat/lng near each other (same city, offset by small amounts) and a price.
3. POST /listings
Body: {title, description, resource_type, price, capacity, lat, lng}
Appends to `listings` with a new integer id. Returns the created listing.
4. GET /listings
Returns the full current `listings` list.
5. POST /match
Body: {description (free text requirement), budget (number), lat, lng, resource_type (optional)}
Logic (implement exactly, this is real computation, not a placeholder):
 - a. Build a TF-IDF matrix using sklearn.feature_extraction.text.TfidfVectorizer over the text corpus formed by [requirement.description] + [listing.description for each listing in listings]. Fit and transform together so vectors are comparable.
 - b. Compute cosine_similarity between the requirement vector (row 0) and every listing vector. This is `semantic\_score` per listing, range 0 to 1.
 - c. Compute `price\_score` per listing:
if budget <= 0: price_score = 0.5
else: price_score = max(0, 1 - abs(listing.price - budget) / budget)
 - d. Compute `distance\_km` using the haversine formula between (lat, lng) of the requirement and each listing. Then `distance\_score` = max(0, 1 - distance_km / 10).
 - e. If resource_type was provided and does not match a listing's resource_type, still include the listing but apply a 0.5x multiplier to its final score (soft filter, not exclusion).
 - f. Compute final_score:
final_score = 0.5 * semantic_score + 0.3 * price_score + 0.2 * distance_score
(weights sum to 1; simplified from the full 5-factor PRD formula since availability and rating aren't part of this MVP's data model)
 - g. Sort listings by final_score descending. Return the top 5 as JSON:

```
{
  "matches": [
    {
      "listing": {...full listing...},
      "final_score": 0.0-1.0 rounded to 3 decimals,
      "breakdown": {
        "semantic": ...,
        "price_fit": ...,
        "distance": ...,
        "distance_km": ...
      }
    },
    ...
  ]
}
```Round all scores to 3 decimal places. Do not return anything not computed by this logic.
6. POST /negotiate
Body: {provider_min, provider_ask, seeker_offer, seeker_max, extra_terms (string, optional)}
Logic (pure deterministic function, implement exactly):
lower_bound = max(provider_min, seeker_offer)
upper_bound = min(provider_ask, seeker_max)
if lower_bound > upper_bound:
return {"status": "no_deal", "message": "No overlapping price range was found between the two parties. Try adjusting the offer or budget."}

```

clearing_price = round((lower_bound + upper_bound) / 2, 2)
summary = f"Settled at Rs.{clearing_price}."
if extra_terms:
    summary += f" Terms: {extra_terms}."
return {"status": "settled", "clearing_price": clearing_price, "summary": summary}
Do not use any language model here. This is a templated string built from real computed
numbers – label it in the UI as "Automated Settlement", not as AI-generated text.

```

7. Add a GET /health endpoint returning {"status": "ok"} for uptime checks.

8. Run with: `uvicorn main:app --host 0.0.0.0 --port $PORT`
 Add a Procfile: `web: uvicorn main:app --host 0.0.0.0 --port $PORT`

```

=====
FRONTEND – single static site, three sections on one page
=====

```

Folder structure:

```

frontend/
  index.html
  style.css
  script.js

```

Design: clean, modern, professional. Color palette: primary navy #3A4876, light lavender accent #C7CEEA, white background, dark gray body text #1A202C. Rounded corners (8px) on cards and buttons. Use a simple sans-serif font stack. Generous padding (at least 16px inside cards). No lorem ipsum, no placeholder images – real labels and real icons/emoji are fine.

At the top of script.js, define:

```

const API_BASE = "PASTE_YOUR_RENDER_BACKEND_URL_HERE";
(leave this as a clearly marked placeholder – the user will fill in their deployed backend URL)

```

Three tabbed sections (simple JS show/hide, not a router):

TAB 1 – "List a Resource" (Provider side)

- A form: title, description (textarea), resource type (select: kitchen/banquet_hall/vehicle/av_equipment/other), price, capacity, and two fields for lat/lng (with a short helper note: "use approximate coordinates, e.g. 19.0760, 72.8777 for Mumbai").
- On submit, POST to /listings, show a success message with the returned listing, and clear the form. Show a loading spinner on the submit button while the request is in flight. On failure, show a clear error message, never fail silently.
- Below the form, show a live list of all current listings (GET /listings on tab load and after every successful submission), rendered as small cards.

TAB 2 – "Find a Resource" (Seeker side)

- A form: free-text description (textarea, placeholder: "e.g. need a banquet hall for 200 guests this Saturday, budget 20000"), budget (number), lat/lng, resource type (optional select).
- Include a "Try a sample search" button that pre-fills the form with a realistic example (e.g., "need a commercial kitchen with ovens for weekend catering", budget 30000, coordinates matching the seeded listings) so the demo always has something meaningful to show.
- On submit, POST to /match, show a loading state, then render the top results as ranked cards: each card shows the listing title, resource type, price, distance in km, the final match percentage as a large number, and the score breakdown (semantic/price/distance) as three small labeled bars or numbers underneath. If no matches are returned, show "No listings available yet – add one in the List a Resource tab first," not a blank screen.
- Each result card has a "Negotiate" button that opens Tab 3 pre-filled with that listing's price as the provider's ask.

TAB 3 – "Negotiate & Settle"

- A form: provider minimum acceptable price, provider ask (pre-filled if arriving from a match card), seeker offer, seeker max budget, extra terms (text, optional).
- On submit, POST to /negotiate. If status is "settled", show the clearing price in large text and the summary sentence below it, in a success-colored card. If "no_deal", show the message in a neutral warning-colored card, not an error color (it's a valid outcome, not a bug).
- Label this section's output clearly as "Automated Settlement Engine" in the UI – do not use the word "AI" here, since this is deterministic arithmetic, not a model.

General frontend requirements:

- Every fetch() call must have a try/catch. On network failure, show "Could not reach the server – check your connection or try again" rather than a blank page or a raw error in the console.
- Every button that triggers a network call must show a disabled/loading state while waiting.
- The page must be usable and legible on a laptop screen at minimum; mobile responsiveness is a nice-to-have, not required for this build.

```

=====
DEPLOYMENT STEPS
=====

```

Backend to Render:

1. Push the backend/ folder to a GitHub repo (or a subfolder of one).
2. On Render.com: New + Web Service, connect the repo, set root directory to backend/ if needed.
3. Build command: `pip install -r requirements.txt`
4. Start command: `uvicorn main:app --host 0.0.0.0 --port $PORT`
5. Choose the free instance type. Deploy. Copy the resulting *.onrender.com URL.
6. IMPORTANT: Render's free tier spins the service down after inactivity, causing a 30-50 second cold start on the first request. Before any live demo or judging, open the /health endpoint in a browser a few minutes beforehand to wake it up.

Frontend to Netlify:

1. Paste the copied Render backend URL into API_BASE in script.js.
2. Drag the frontend/ folder directly into Netlify's "Deploy manually" drop zone (fastest path, no git or build step needed), or connect the repo with an empty build command and frontend/ as the publish directory.
3. Confirm the live Netlify URL loads and successfully calls the backend (test all three tabs).

=====

FINAL CHECKLIST BEFORE CALLING THIS DONE

=====

- [] Adding a listing actually appears in the listings list afterward (real state, not mocked)
- [] Running a match with two different free-text descriptions produces two different rankings (proves the TF-IDF similarity is actually running on the input, not fixed)
- [] Changing the budget changes the price_fit score and can change the ranking order
- [] Negotiation with non-overlapping ranges correctly returns "no_deal", not a fake price
- [] Negotiation with overlapping ranges returns the exact midpoint math, verify by hand once
- [] No console errors on any tab
- [] No placeholder/lorem text remains anywhere in the shipped UI
- [] The backend URL in script.js is the real deployed URL, not localhost

Notes For Judges, Keep In Your Back Pocket

- If asked why TF-IDF instead of a transformer embedding model: it was scoped for free-tier hosting constraints, and the architecture supports swapping in a transformer embedding model without changing anything else in the system.
- The negotiation engine was never meant to involve a language model in the first place. Keeping arithmetic out of AI hands entirely was a deliberate design decision from the original PRD, and this build is the same decision, applied honestly under a tighter timeline.